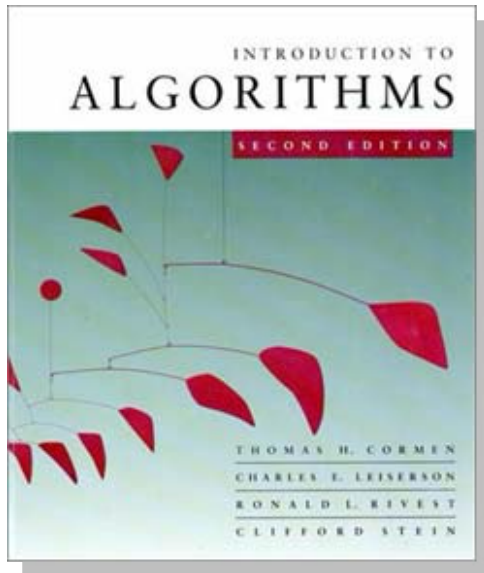


Introduction to Algorithms

6.046J/18.401J



LECTURE 5

Sorting Lower Bounds

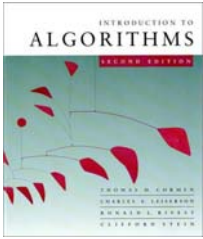
- Decision trees

Linear-Time Sorting

- Counting sort
- Radix sort

Appendix: Punched cards

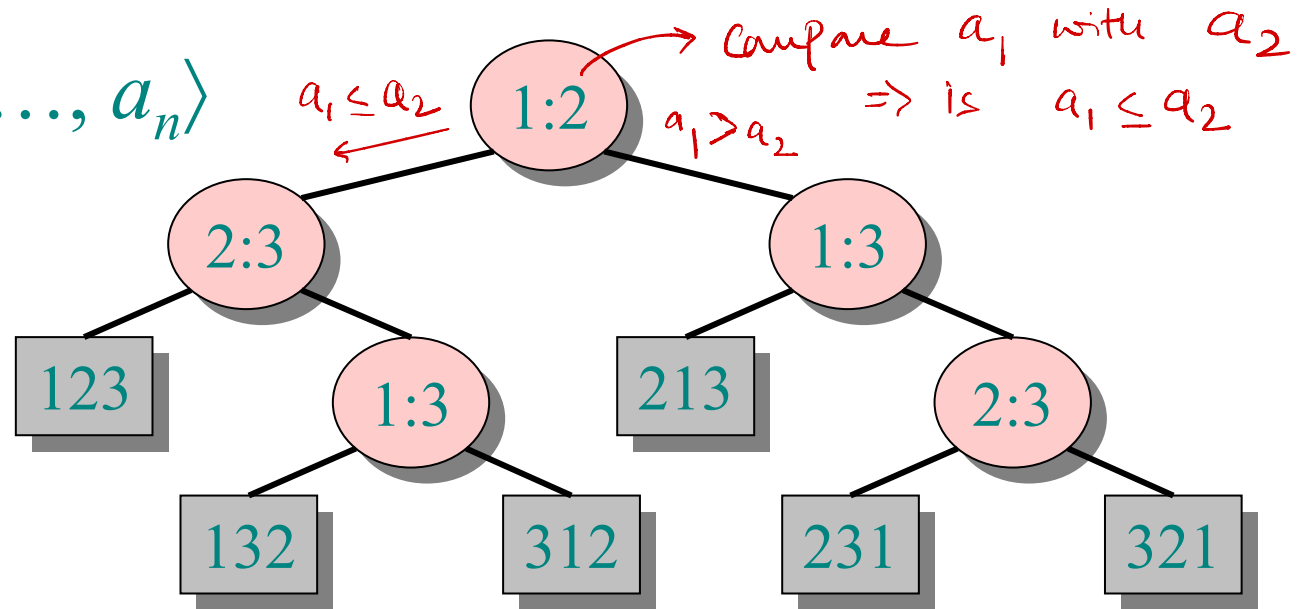
Prof. Erik Demaine



Decision-tree example

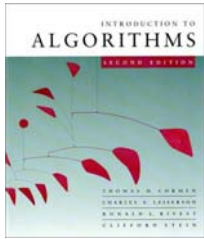
Sort $\langle a_1, a_2, \dots, a_n \rangle$

$i:j$
 $\Rightarrow$ is $a_i \leq a_j$



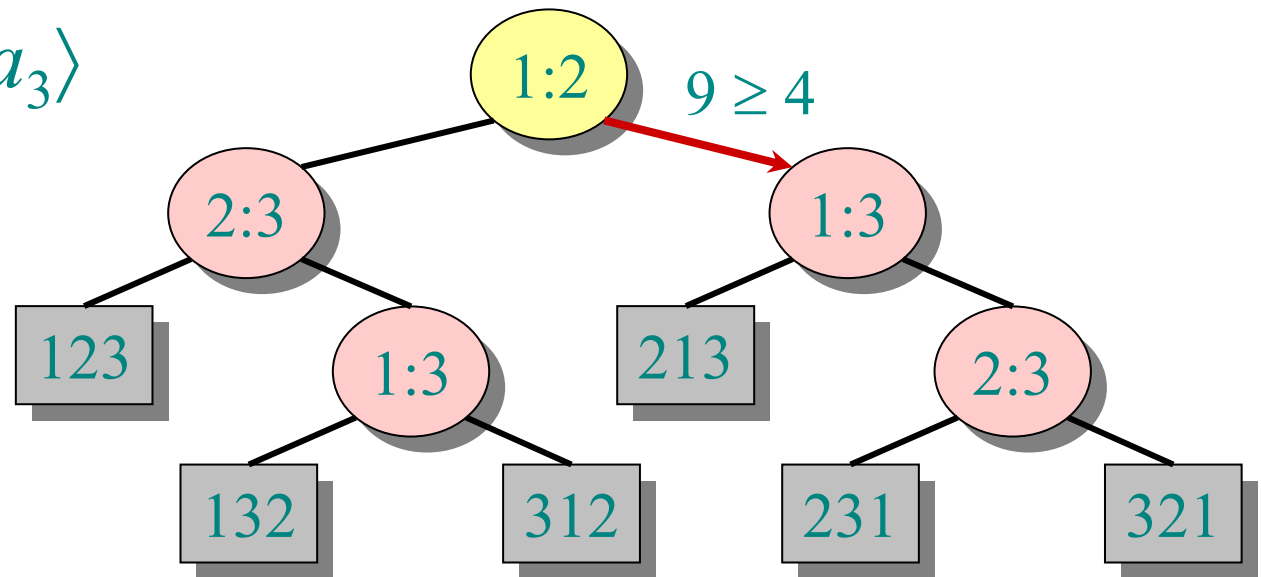
Each internal node is labeled $i:j$ for $i, j \in \{1, 2, \dots, n\}$.

- The left subtree shows subsequent comparisons if $a_i \leq a_j$.
- The right subtree shows subsequent comparisons if $a_i \geq a_j$.



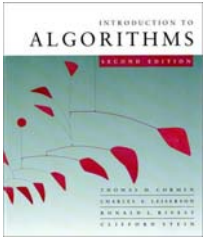
Decision-tree example

Sort $\langle a_1, a_2, a_3 \rangle$
 $= \langle 9, 4, 6 \rangle$:



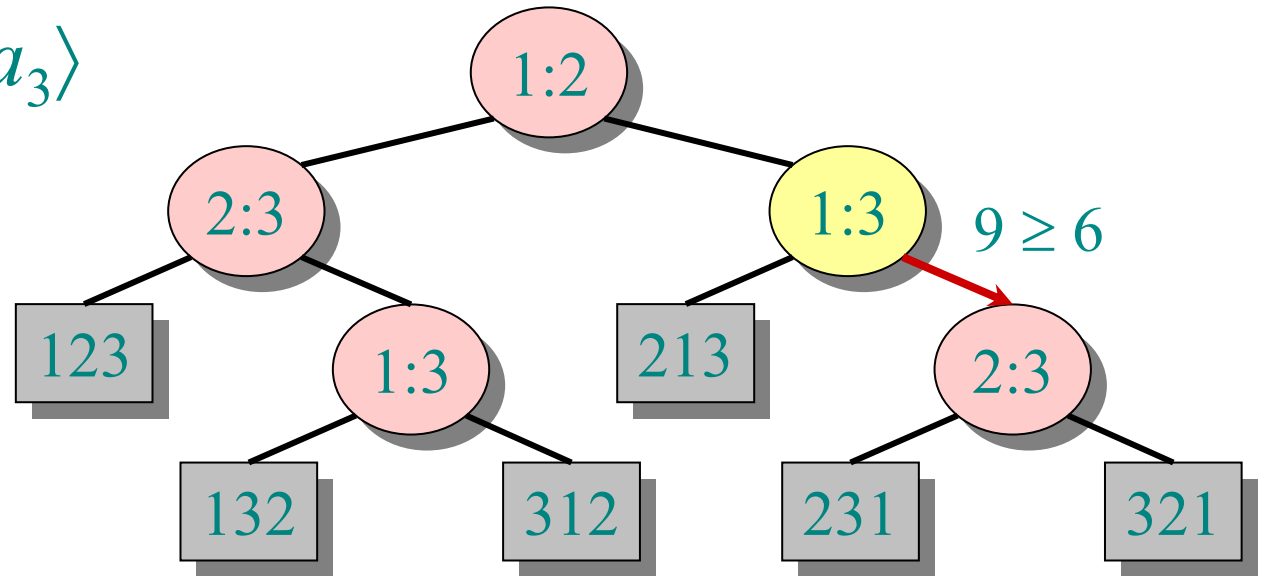
Each internal node is labeled $i:j$ for $i, j \in \{1, 2, \dots, n\}$.

- The left subtree shows subsequent comparisons if $a_i \leq a_j$.
- The right subtree shows subsequent comparisons if $a_i \geq a_j$.



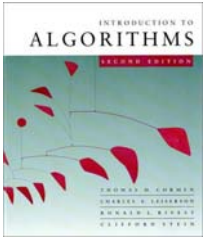
Decision-tree example

Sort $\langle a_1, a_2, a_3 \rangle$
 $= \langle 9, 4, 6 \rangle$:



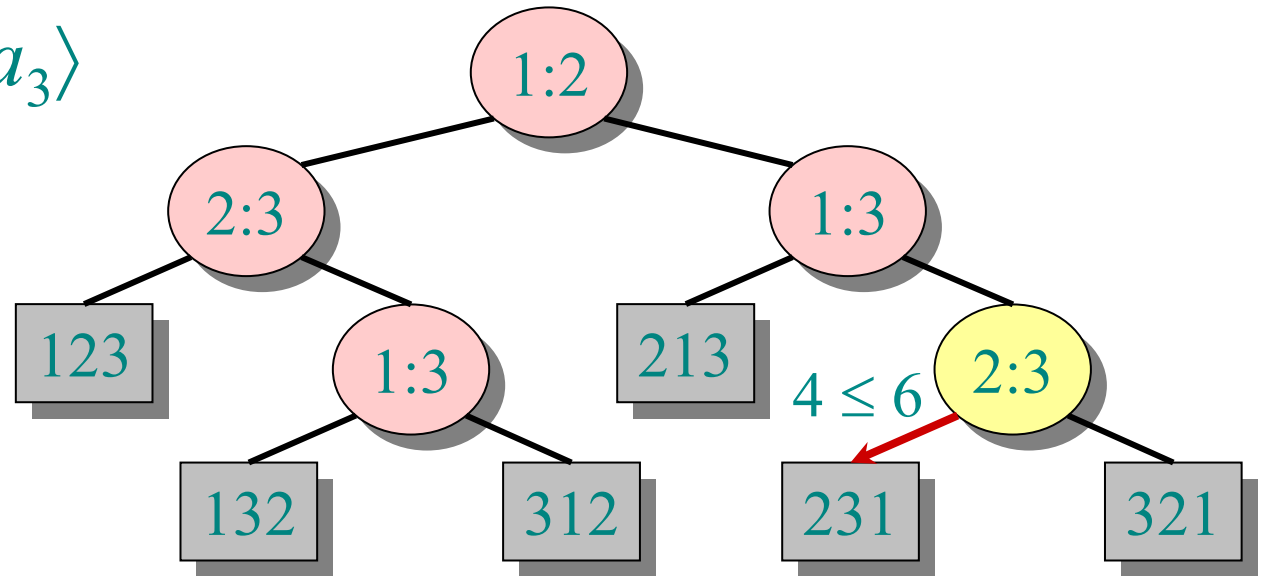
Each internal node is labeled $i:j$ for $i, j \in \{1, 2, \dots, n\}$.

- The left subtree shows subsequent comparisons if $a_i \leq a_j$.
- The right subtree shows subsequent comparisons if $a_i \geq a_j$.



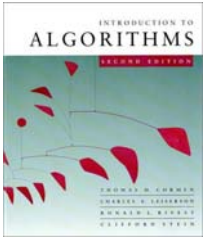
Decision-tree example

Sort $\langle a_1, a_2, a_3 \rangle$
 $= \langle 9, 4, 6 \rangle$:



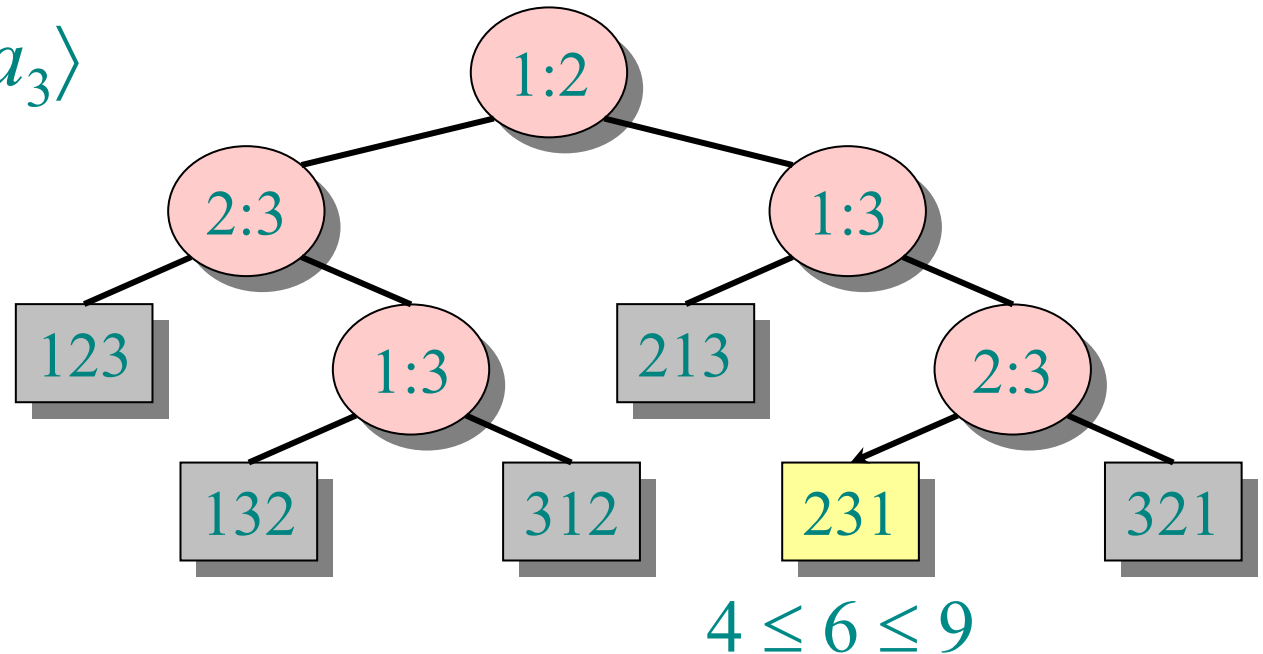
Each internal node is labeled $i:j$ for $i, j \in \{1, 2, \dots, n\}$.

- The left subtree shows subsequent comparisons if $a_i \leq a_j$.
- The right subtree shows subsequent comparisons if $a_i \geq a_j$.

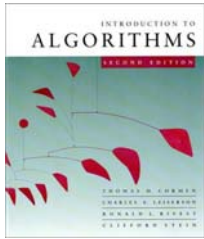


Decision-tree example

Sort $\langle a_1, a_2, a_3 \rangle$
 $= \langle 9, 4, 6 \rangle$:

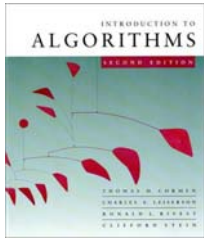


Each leaf contains a permutation $\langle \pi(1), \pi(2), \dots, \pi(n) \rangle$ to indicate that the ordering $a_{\pi(1)} \leq a_{\pi(2)} \leq \dots \leq a_{\pi(n)}$ has been established.



Lower bound for comparison sorting

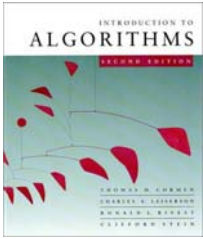
Corollary. Heapsort and merge sort are asymptotically optimal comparison sorting algorithms. 



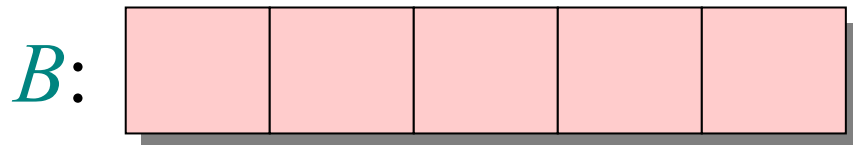
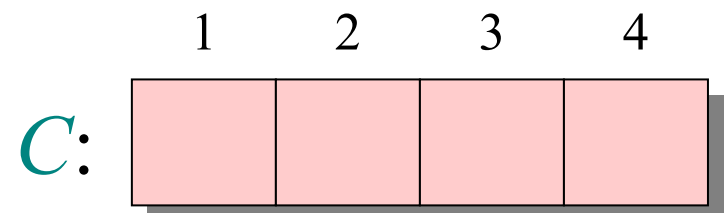
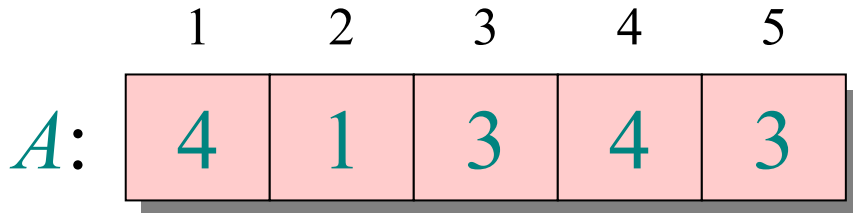
Sorting in linear time

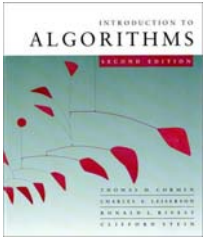
Counting sort: No comparisons between elements.

- **Input:** $A[1 \dots n]$, where $A[j] \in \{1, 2, \dots, k\}$.
- **Output:** $B[1 \dots n]$, sorted.
- **Auxiliary storage:** $C[1 \dots k]$.



Counting-sort example





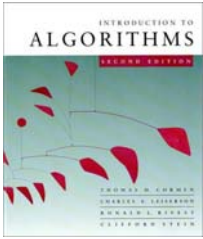
Loop 1

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

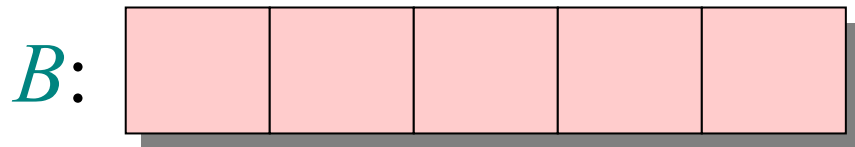
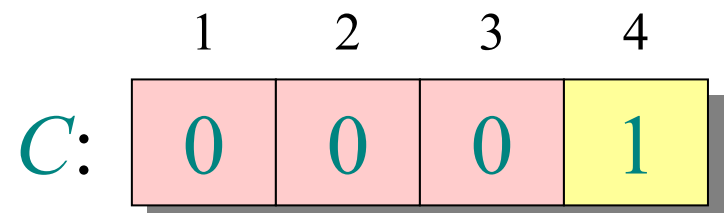
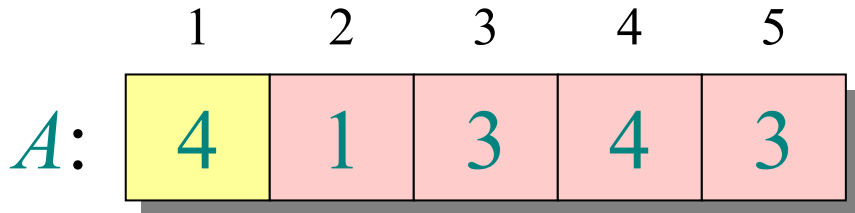
	1	2	3	4
<i>C</i> :	0	0	0	0

<i>B</i> :					
------------	--	--	--	--	--

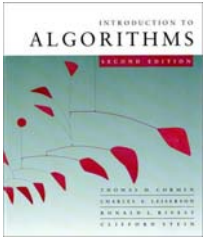
for $i \leftarrow 1$ **to** k
 do $C[i] \leftarrow 0$



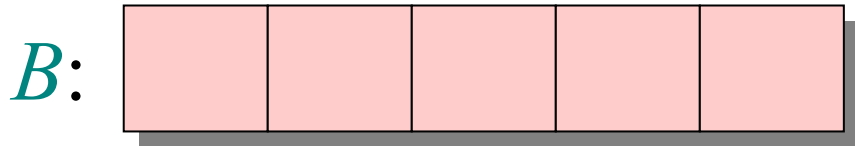
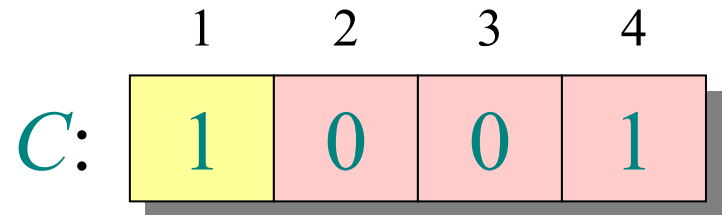
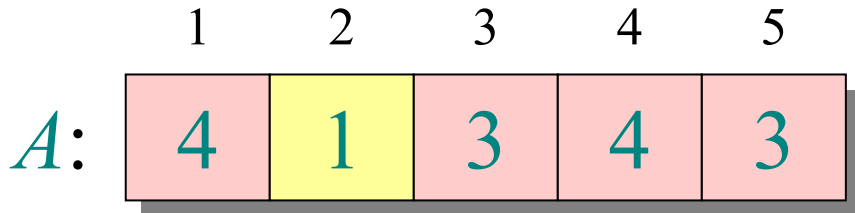
Loop 2



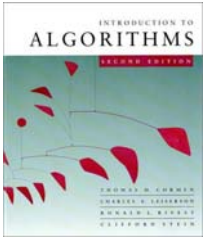
for $j \leftarrow 1$ **to** n
 do $C[A[j]] \leftarrow C[A[j]] + 1$ $\triangleright C[i] = |\{\text{key} = i\}|$



Loop 2



for $j \leftarrow 1$ **to** n
 do $C[A[j]] \leftarrow C[A[j]] + 1 \quad \triangleright C[i] = |\{\text{key} = i\}|$



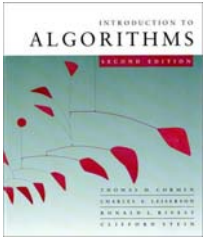
Loop 2

	1	2	3	4	5
A :	4	1	3	4	3

	1	2	3	4
C :	1	0	1	1

B :					
-------	--	--	--	--	--

for $j \leftarrow 1$ **to** n
 do $C[A[j]] \leftarrow C[A[j]] + 1$ $\triangleright C[i] = |\{\text{key} = i\}|$



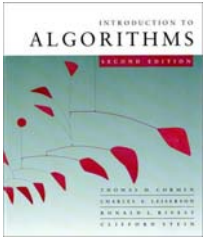
Loop 2

	1	2	3	4	5
A :	4	1	3	4	3

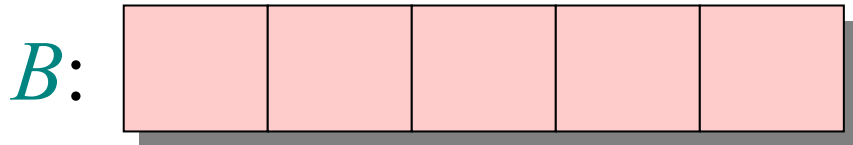
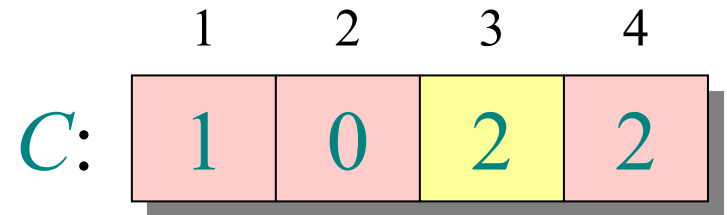
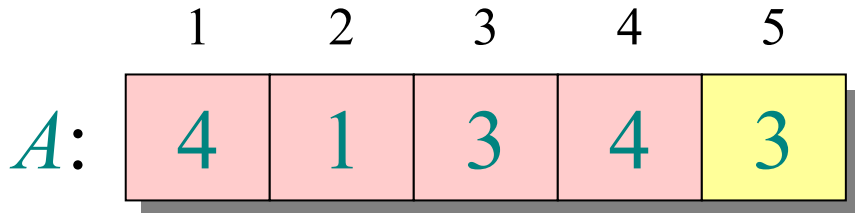
	1	2	3	4
C :	1	0	1	2

B :					
-------	--	--	--	--	--

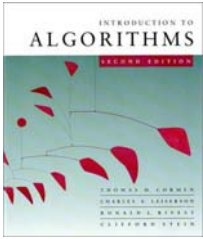
for $j \leftarrow 1$ **to** n
 do $C[A[j]] \leftarrow C[A[j]] + 1$ $\triangleright C[i] = |\{\text{key} = i\}|$



Loop 2



for $j \leftarrow 1$ **to** n
 do $C[A[j]] \leftarrow C[A[j]] + 1$ $\triangleright C[i] = |\{\text{key} = i\}|$



Loop 3

	1	2	3	4	5
A :	4	1	3	4	3

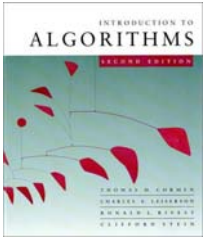
B :					
-------	--	--	--	--	--

	1	2	3	4
C :	1	0	2	2

C' :	1	1	2	2
--------	---	---	---	---

for $i \leftarrow 2$ **to** k

do $C[i] \leftarrow C[i] + C[i-1]$ $\triangleright C[i] = |\{\text{key} \leq i\}|$



Loop 3

	1	2	3	4	5
A :	4	1	3	4	3

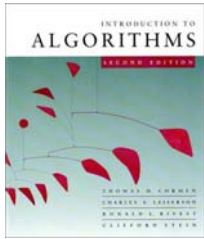
B :					
-------	--	--	--	--	--

	1	2	3	4
C :	1	0	2	2

C' :	1	1	3	2
--------	---	---	---	---

for $i \leftarrow 2$ **to** k

do $C[i] \leftarrow C[i] + C[i-1]$ $\triangleright C[i] = |\{\text{key} \leq i\}|$



Loop 3

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

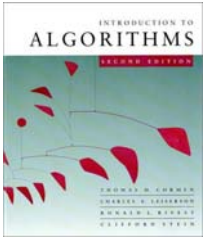
<i>B</i> :					
------------	--	--	--	--	--

	1	2	3	4
<i>C</i> :	1	0	2	2

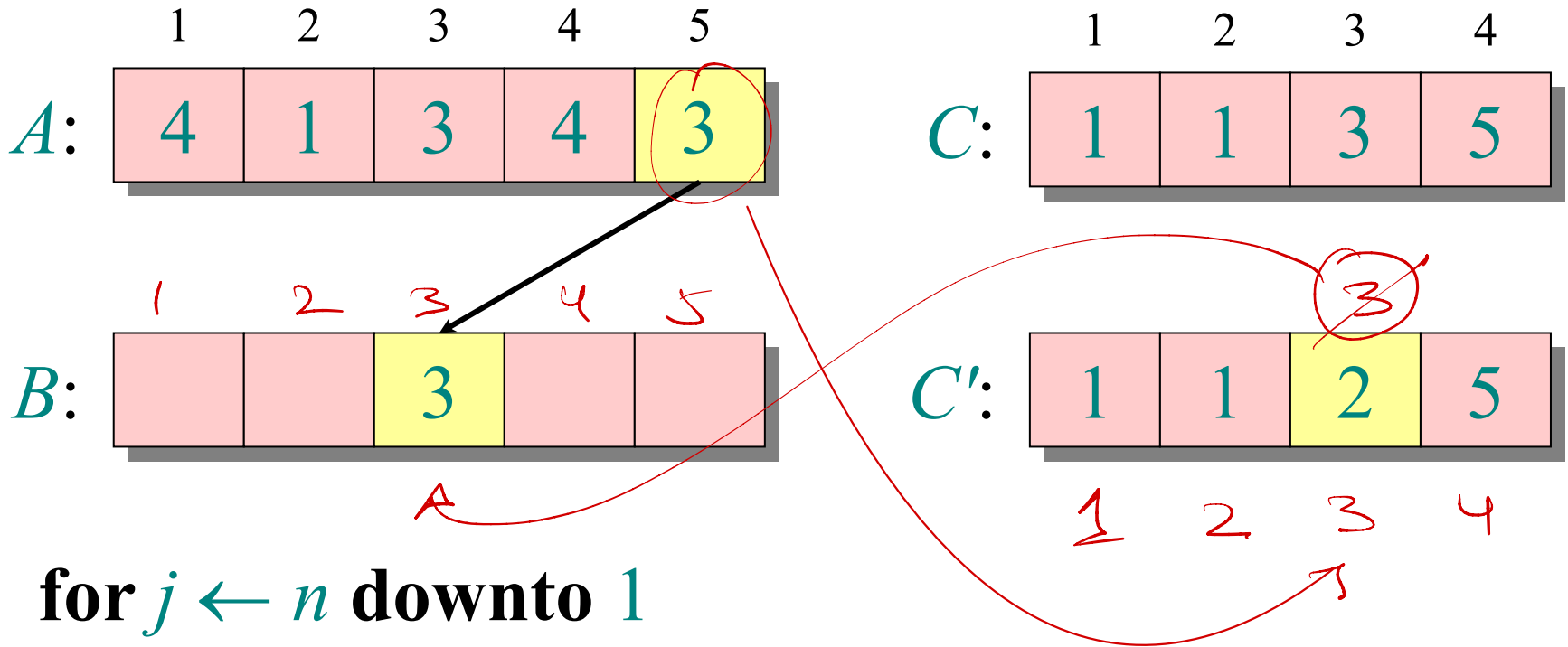
<i>C'</i> :	1	1	3	5
-------------	---	---	---	---

for $i \leftarrow 2$ **to** k

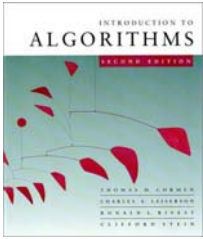
do $C[i] \leftarrow C[i] + C[i-1]$ $\triangleright C[i] = |\{\text{key} \leq i\}|$



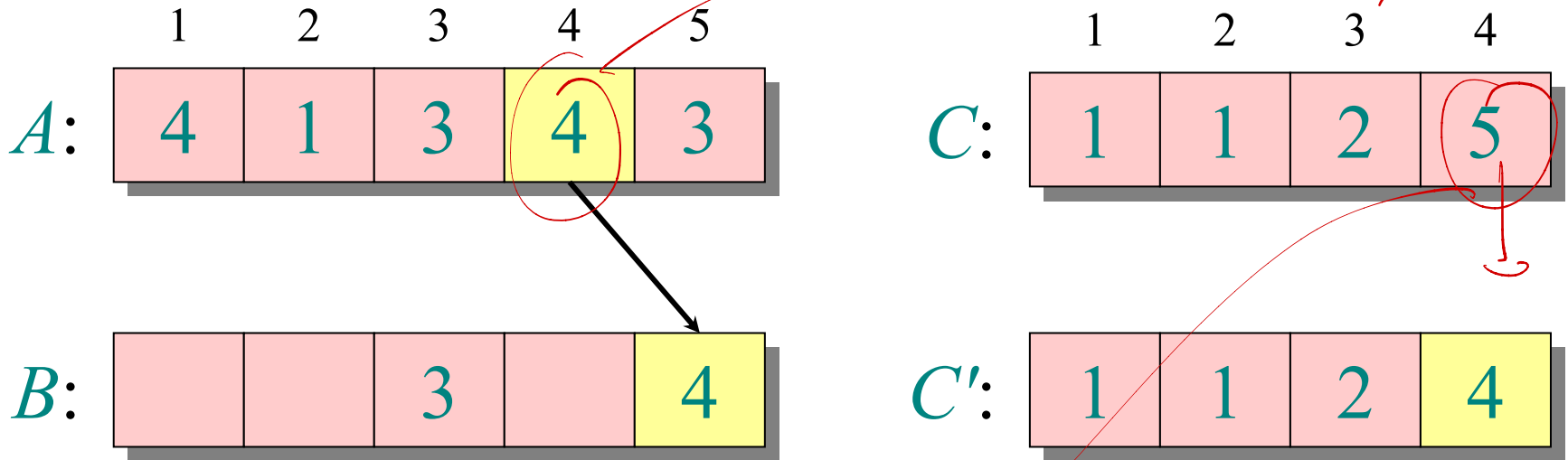
Loop 4



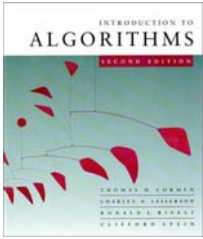
```
for  $j \leftarrow n$  downto 1
  do  $B[C[A[j]]] \leftarrow A[j]$ 
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```



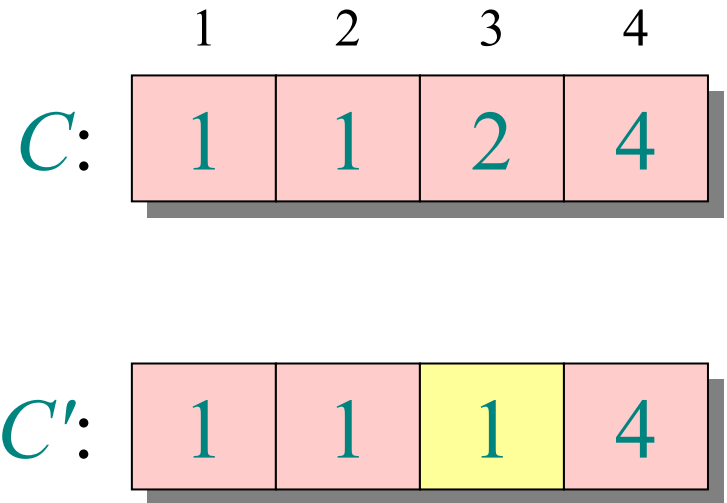
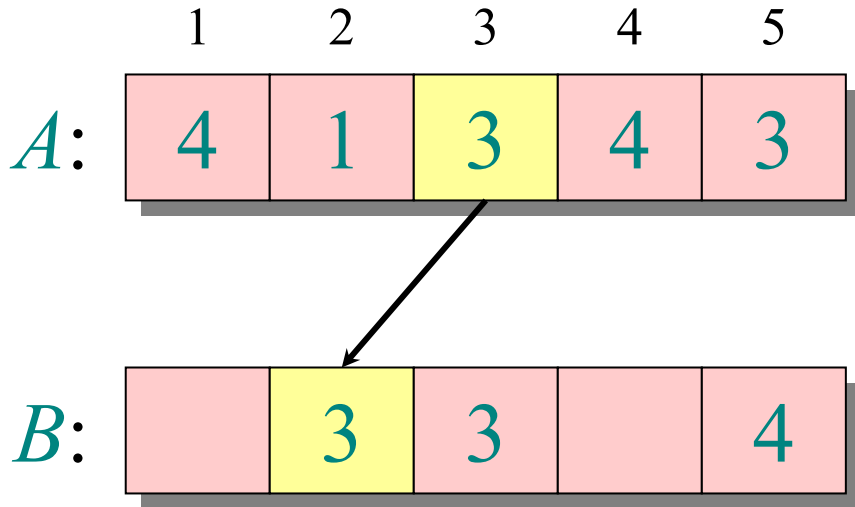
Loop 4



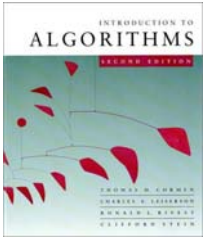
```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```



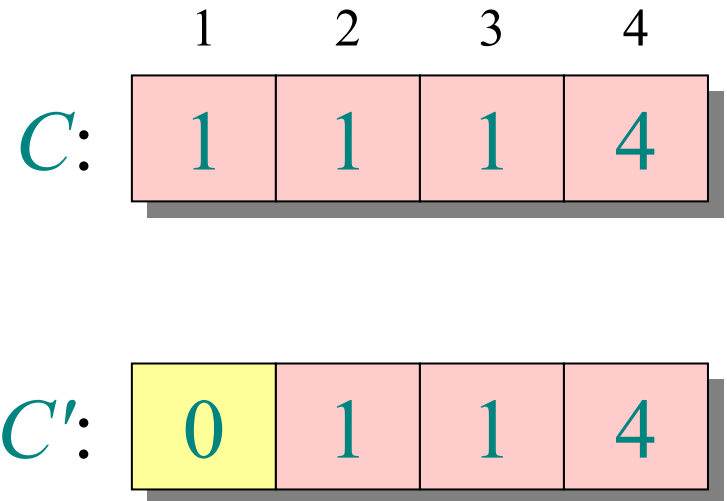
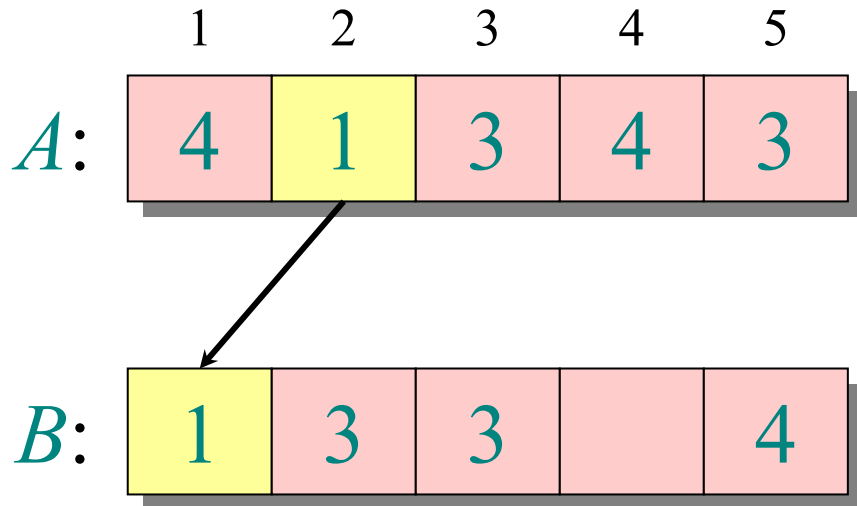
Loop 4



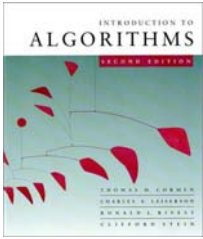
```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```



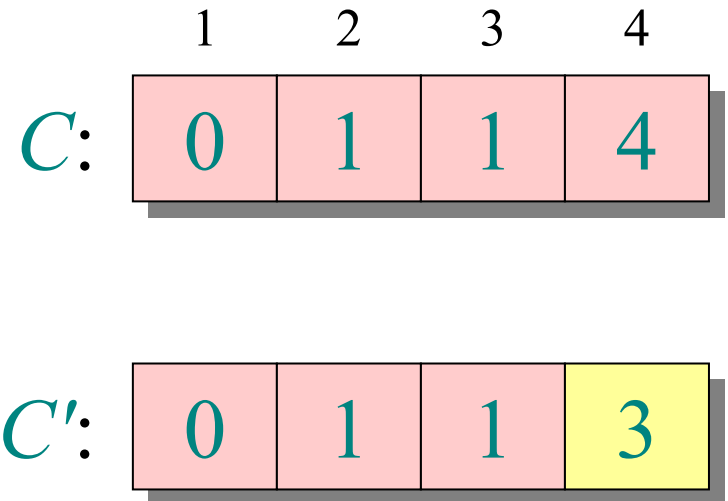
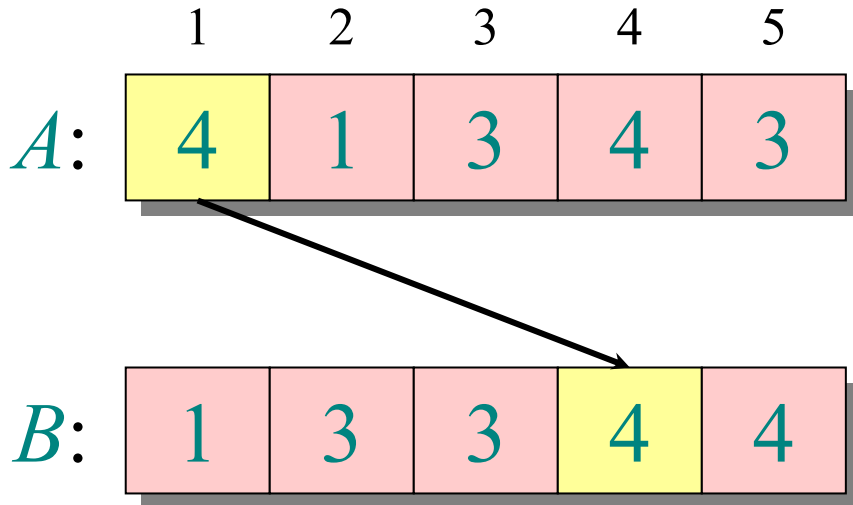
Loop 4



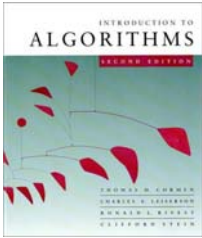
```
for  $j \leftarrow n$  downto 1
  do  $B[C[A[j]]] \leftarrow A[j]$ 
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```



Loop 4



```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```



Analysis

$\Theta(k)$ { for $i \leftarrow 1$ to k
do $C[i] \leftarrow 0$

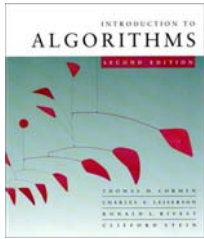
$\Theta(n)$ { for $j \leftarrow 1$ to n
do $C[A[j]] \leftarrow C[A[j]] + 1$

$\Theta(k)$ { for $i \leftarrow 2$ to k
do $C[i] \leftarrow C[i] + C[i-1]$

$\Theta(n)$ { for $j \leftarrow n$ downto 1
do $B[C[A[j]]] \leftarrow A[j]$
 $C[A[j]] \leftarrow C[A[j]] - 1$

$\Theta(n + k)$

if $k \ll n$ then $T(n) = \Theta(n)$!



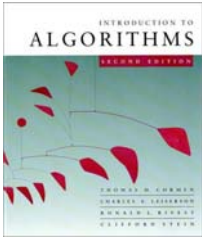
Running time

If $k = O(n)$, then counting sort takes $\Theta(n)$ time.

- But, sorting takes $\Omega(n \lg n)$ time!
- Where's the fallacy?

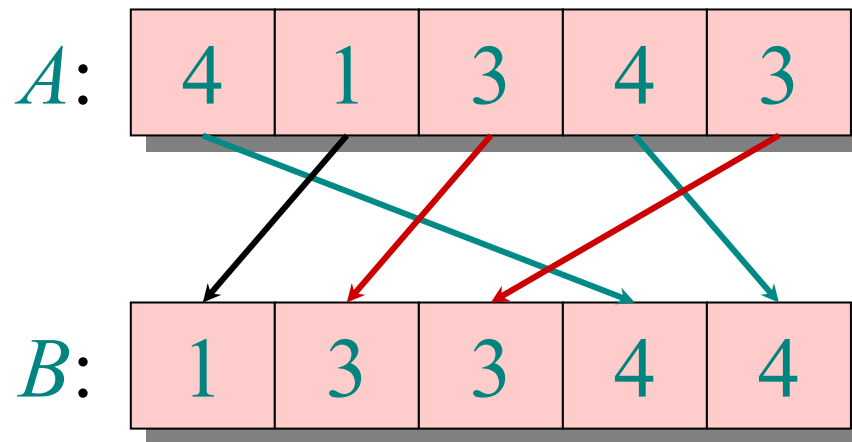
Answer:

- *Comparison sorting* takes $\Omega(n \lg n)$ time.
- Counting sort is not a *comparison sort*.
- In fact, not a single comparison between elements occurs!

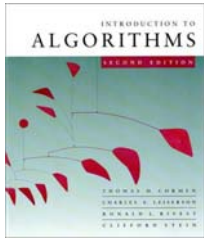


Stable sorting

Counting sort is a *stable* sort: it preserves the input order among equal elements.

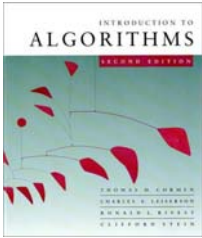


Exercise: What other sorts have this property?

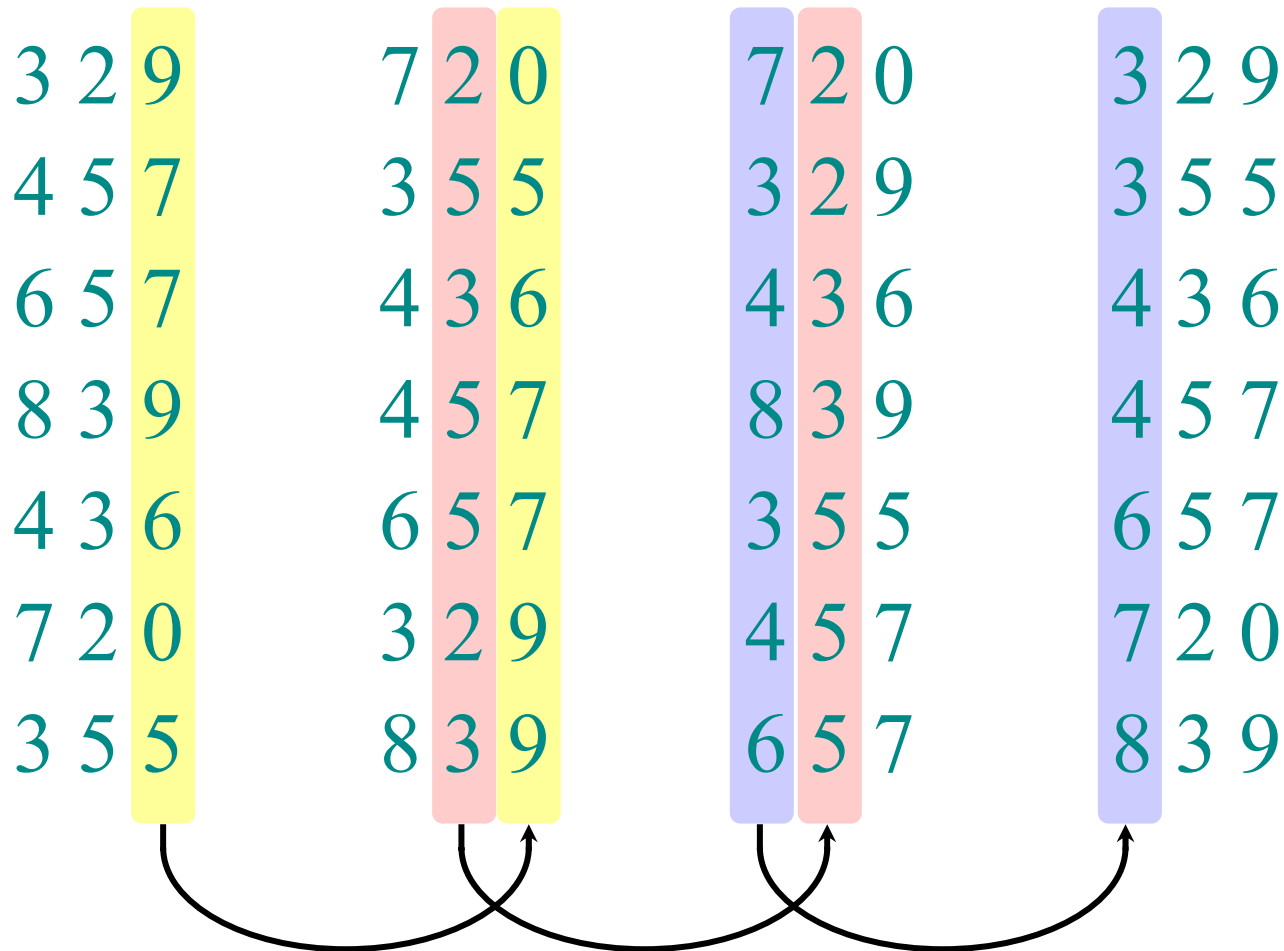


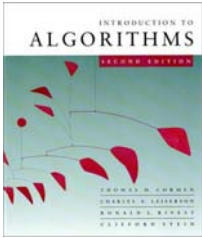
Radix sort

- *Origin*: Herman Hollerith's card-sorting machine for the 1890 U.S. Census. (See Appendix .)
- Digit-by-digit sort.
- Hollerith's original (bad) idea: sort on most-significant digit first.
- Good idea: Sort on *least-significant digit first* with auxiliary *stable* sort.



Operation of radix sort





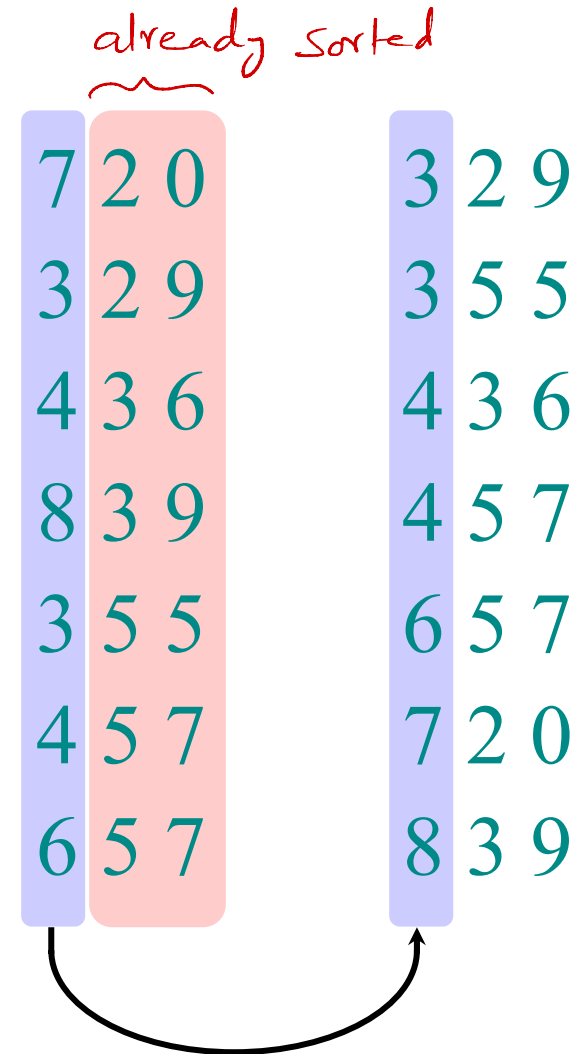
Correctness of radix sort

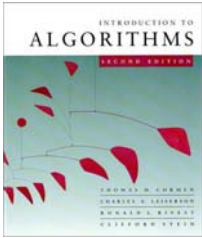
Induction on digit position

- Assume that the numbers are sorted by their low-order $t - 1$ digits.

- Sort on digit t

given that we use
a stable sort.

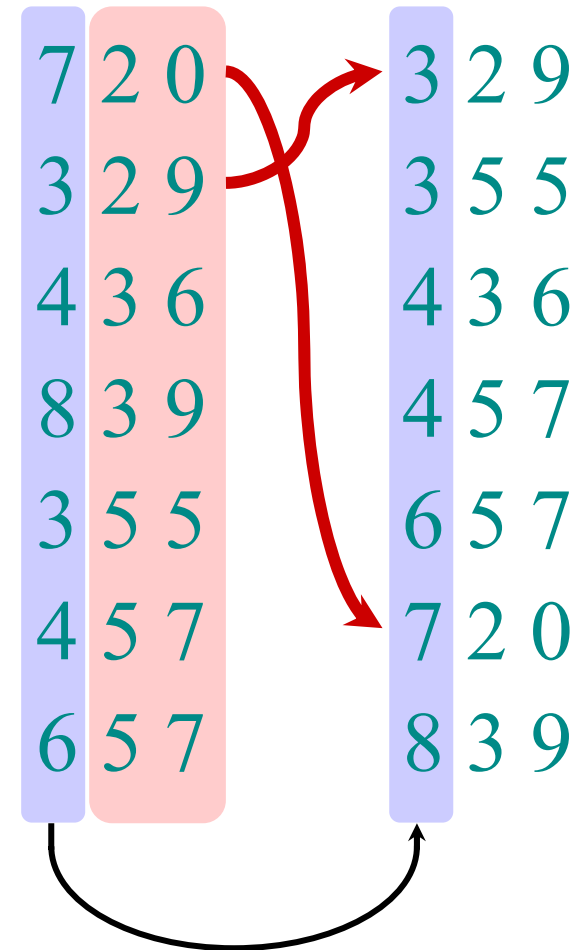


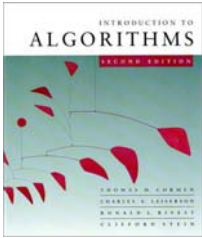


Correctness of radix sort

Induction on digit position

- Assume that the numbers are sorted by their low-order $t - 1$ digits.
- Sort on digit t
 - Two numbers that differ in digit t are correctly sorted.

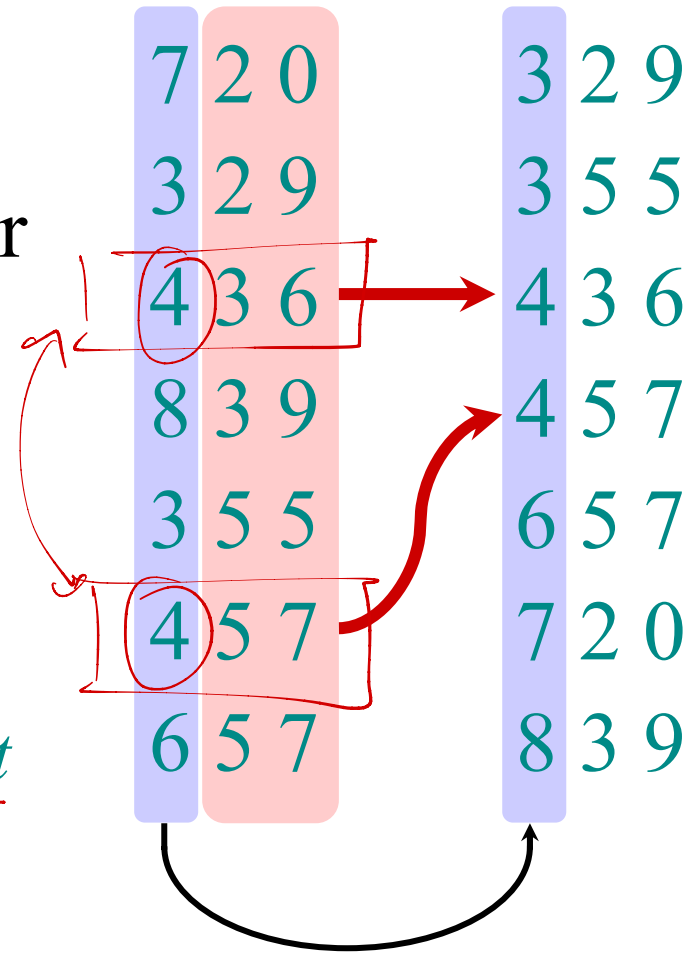




Correctness of radix sort

Induction on digit position

- Assume that the numbers are sorted by their low-order $t - 1$ digits.
- Sort on digit t
 - Two numbers that differ in digit t are correctly sorted.
 - Two numbers equal in digit t are put in the same order as the input $\Rightarrow$ correct order.



$\rightarrow$ Since we use stable sorting!

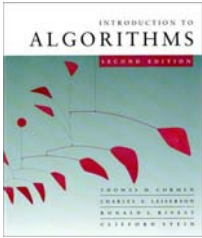
Complexity of radix sort:

→ Each digit lies between 0 & $k-1$

(for decimal digits: $[0, 9]$)

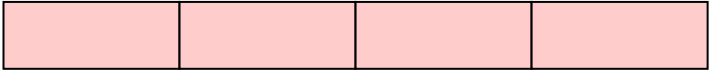
→ Each pass over n d -digit numbers
takes $\theta(n+k)$

→ for d -passes, total time = $\theta(d(n+k))$



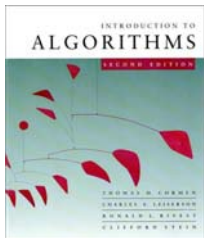
Analysis of radix sort

- Assume counting sort is the auxiliary stable sort.
- Sort n computer words of b bits each.
- Each word can be viewed as having b/r base- 2^r digits.

Example: 32-bit word  base-256 number

$r = 8 \Rightarrow b/r = 4$ passes of counting sort on base- 2^8 digits; or $r = 16 \Rightarrow b/r = 2$ passes of counting sort on base- 2^{16} digits.

How many passes should we make?



Analysis (continued)

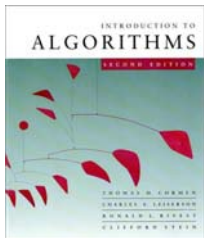
Recall: Counting sort takes $\Theta(n + k)$ time to sort n numbers in the range from 0 to $k - 1$.

If each b -bit word is broken into r -bit pieces, each pass of counting sort takes $\Theta(n + 2^r)$ time. Since there are b/r passes, we have

$$T(n, b) = \Theta\left(\frac{b}{r}(n + 2^r)\right).$$

Choose r to minimize $T(n, b)$:

- Increasing r means fewer passes, but as $r \gg \lg n$, the time grows exponentially.



Choosing r

$$T(n, b) = \Theta\left(\frac{b}{r} (n + 2^r)\right)$$

Minimize $T(n, b)$ by differentiating and setting to 0.

Or, just observe that we don't want $2^r \gg n$, and there's no harm asymptotically in choosing r as large as possible subject to this constraint.

Choosing $r = \lg n$ implies $T(n, b) = \Theta(bn/\lg n)$.

- For numbers in the range from 0 to $n^d - 1$, we have $b = d \lg n \Rightarrow$ radix sort runs in $\Theta(dn)$ time.

Bucket Sort

- Assumes input is drawn from a uniform distribution
- Avg running time of $O(n)$
- Input number in range $[0, 1]$
(input sampled from uniform distribution)

Approach :

- (1) Divide the interval $[0, 1)$ into n equal sized buckets
- (2) Distribute the n input numbers into the buckets

[Note: we do not expect many numbers falling into each bucket due to uniform and independent distribution over $[0, 1)$]

(3) To produce output, we simply sort each bucket and go through each bucket in order, listing elements in each.

— Given input array $A[1:n]$ → Use any sorting algo. e.g. Insertion sort $O(n^2)$!

where $A[i]$ satisfies $0 \leq A[i] < 1$

create an array $B[0:n-1]$ of linked lists (buckets)

Bucket - Sort (A, n)

Let $B[0:n-1]$ be a new array
for $i = 0$ to $n-1$

make $B[i]$ an empty linked list

for $i = 1$ to n

insert $A[i]$ into list $B[\lfloor n \cdot A[i] \rfloor]$

for $i = 0$ to $n-1$

sort list $B[i]$ with insertion sort

Concatenate lists $B[0], B[1], \dots, B[n-1]$ together in order

Return concatenated list

Correctness : Consider two elements $A[i]$ and $A[j]$ such that

$$A[i] \leq A[j]$$

— Since $\lfloor n A[i] \rfloor \leq \lfloor n A[j] \rfloor$, either $A[i]$ goes into the same bucket as $A[j]$, or into a bucket with lower index.

— If both go into the same bucket then insertion sort puts them into correct order.

— If they go into different buckets then the Concatenation puts them into correct order.

Running-time complexity : All executions take $O(n)$ except
insertion sort

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2) \quad , \text{ where } n_i \text{ is the number of element in } i^{\text{th}} \text{ bucket.}$$

To calculate average-case running-time, take expectation.

Let n_i be the random variable denoting the number of elements placed in bucket $B[i]$

$$\begin{aligned} E(T(n)) &= E\left[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)\right] \\ &= \Theta(n) + \sum_{i=0}^{n-1} E[O(n_i^2)] \quad \text{--- linearity of expectation} \\ &= \Theta(n) + \sum_{i=0}^{n-1} O(E[n_i^2]) \quad \text{--- } E[aX] = aE[X] \end{aligned}$$

Claim: $E[n_i^2] = (2 - 1/n)$

- for each $i = 0, 1, 2, \dots, (n-1)$, each bucket i has the same value of $E[n_i^2]$ since each value from A is equally likely to fall into a bucket.

— View each random variable n_i as the number of successes in n Bernaulli trials.

Success = When element goes into bucket $B[i]$

$$\begin{cases} p = \frac{1}{n} & \text{for success} \\ q = 1 - \frac{1}{n} & \text{for failure} \end{cases}$$

$$E[n_i] = np = n\left(\frac{1}{n}\right) = 1$$

$$\text{Var}[n_i] = npq = \left(1 - \frac{1}{n}\right)$$

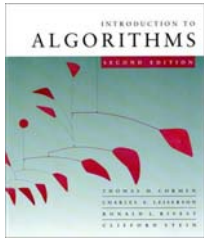
$$\begin{aligned} E[n_i^2] &= \text{Var}[n_i] + E^2[n_i] \\ &= \left(1 - \frac{1}{n}\right) + 1^2 = \left(2 - \frac{1}{n}\right) \end{aligned}$$



$\Rightarrow$ Avg. case running time of bucket sort is :

$$T(n) = \Theta(n) + n \cdot O\left(2 - \frac{1}{n}\right)$$

$$\Rightarrow \boxed{T(n) = \Theta(n)}$$



Conclusions

In practice, radix sort is fast for large inputs, as well as simple to code and maintain.

Example (32-bit numbers):

- At most 3 passes when sorting ≥ 2000 numbers.
- Merge sort and quicksort do at least $\lceil \lg 2000 \rceil = 11$ passes.

Downside: Unlike quicksort, radix sort displays little locality of reference, and thus a well-tuned quicksort fares better on modern processors, which feature steep memory hierarchies.